

Neural networks and CNN

Biplab Banerjee

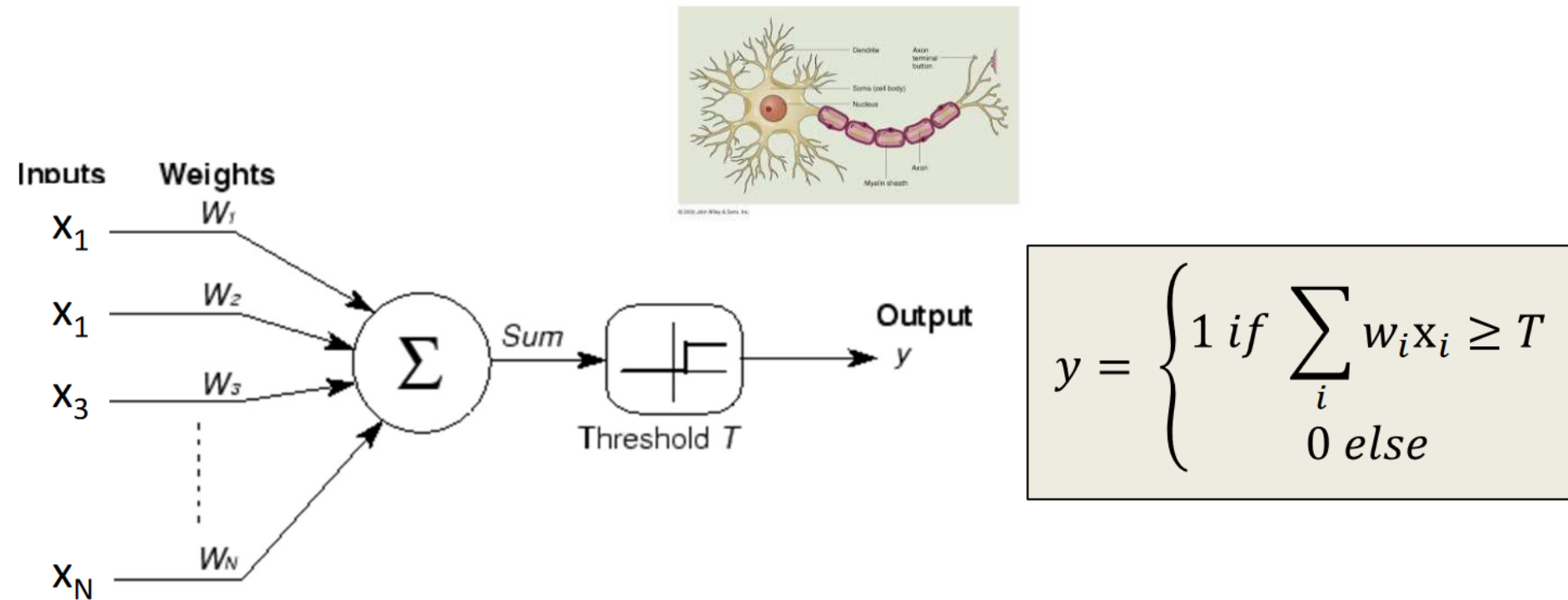
Thanks to towardsdatascience, Vincente Ordonez

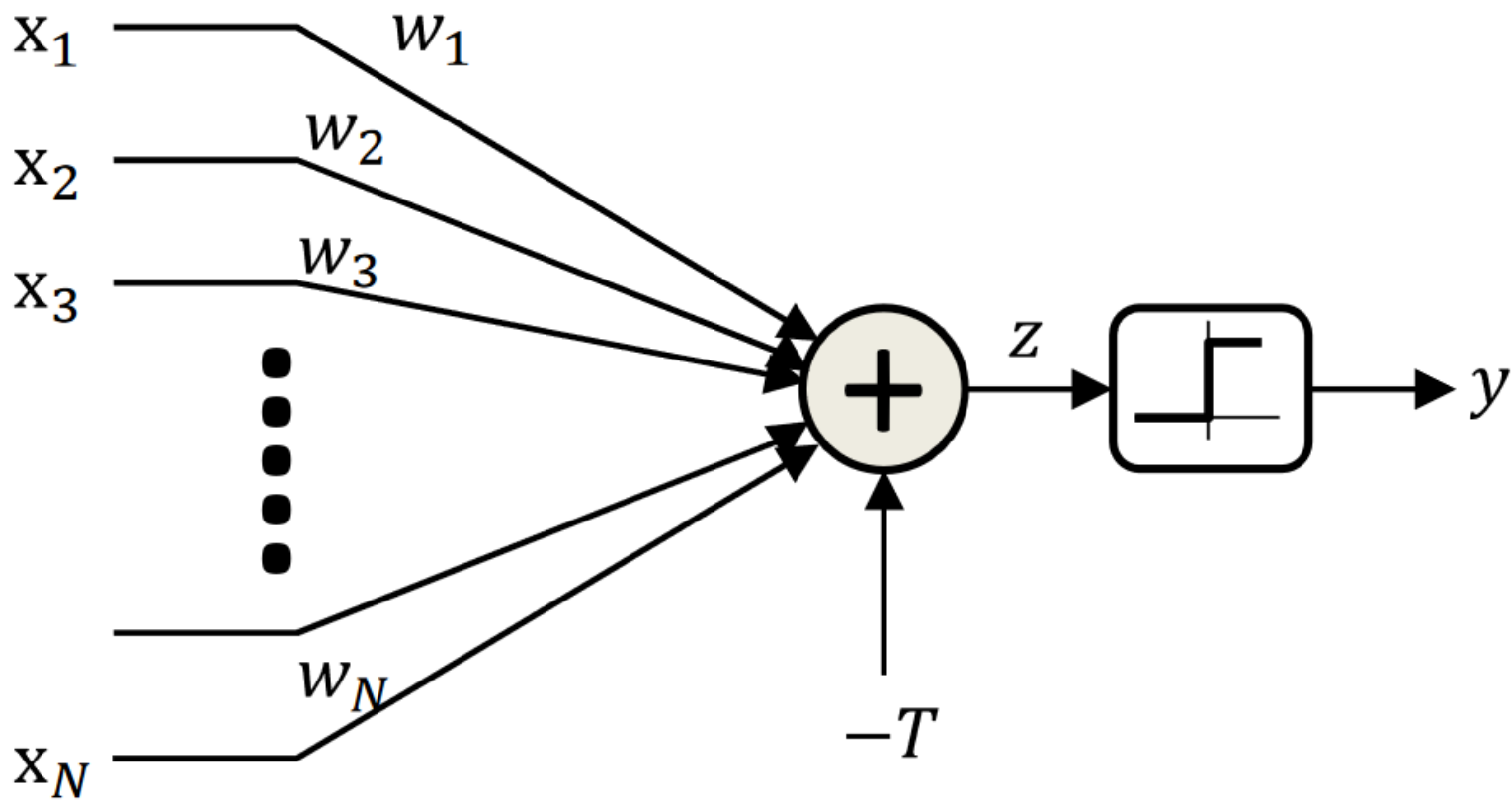
This file is meant for personal use by mepravinpatil@gmail.com only.

Sharing or publishing the contents in part or full is liable for legal action.

Perceptron Model

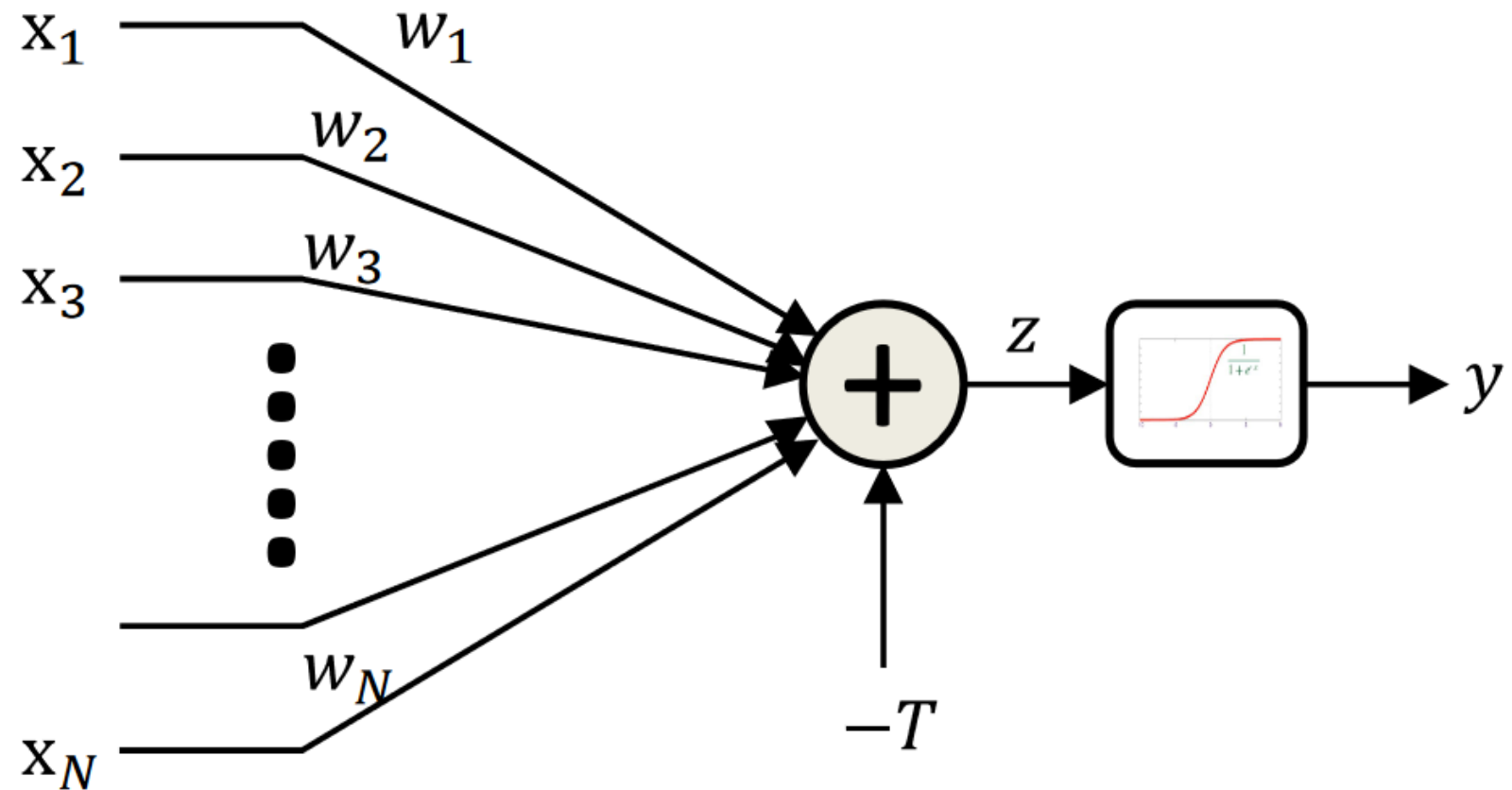
Frank Rosenblatt (1957) - Cornell University





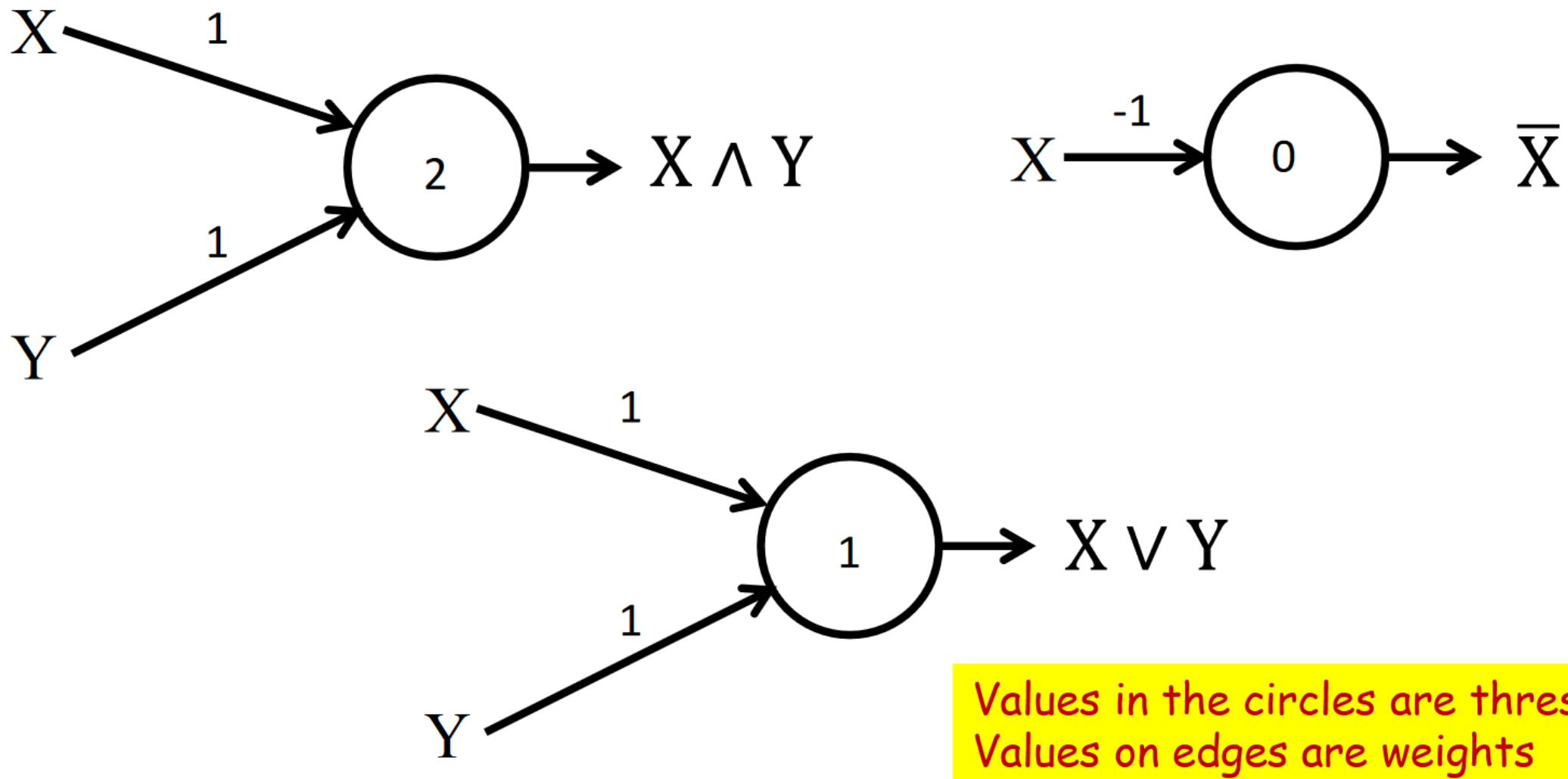
$$z = \sum_i w_i x_i - T$$

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{else} \end{cases}$$



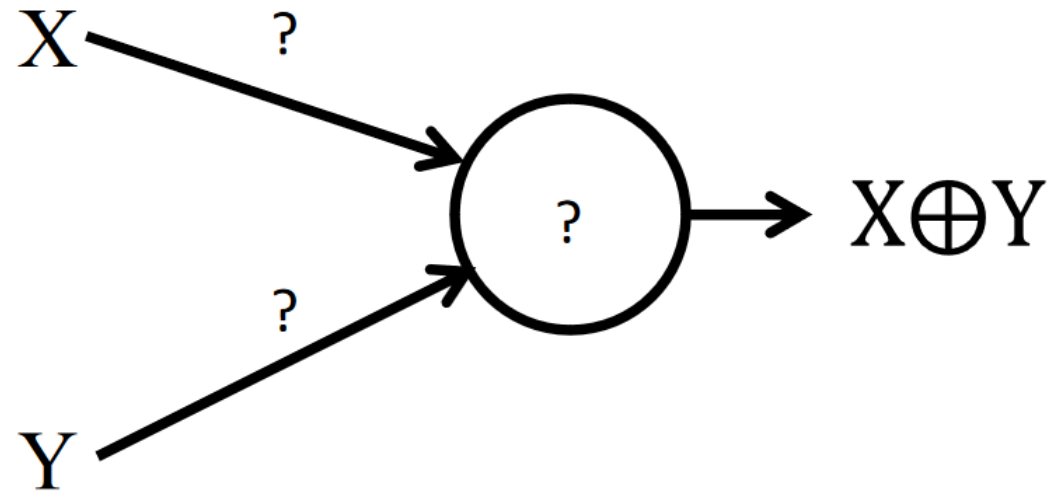
$$z = \sum_i w_i x_i - T$$

$$y = \frac{1}{1 + \exp(-z)}$$

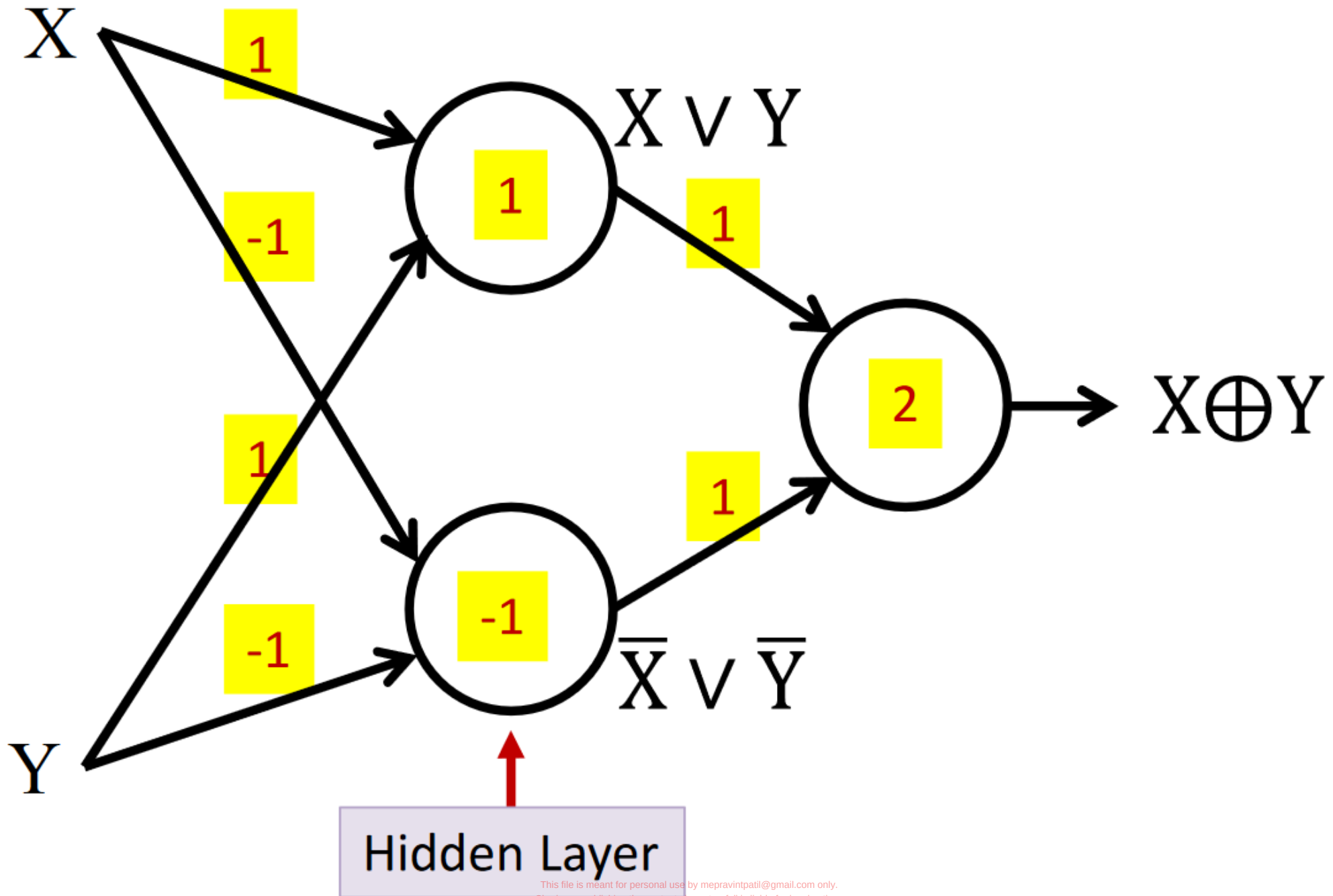


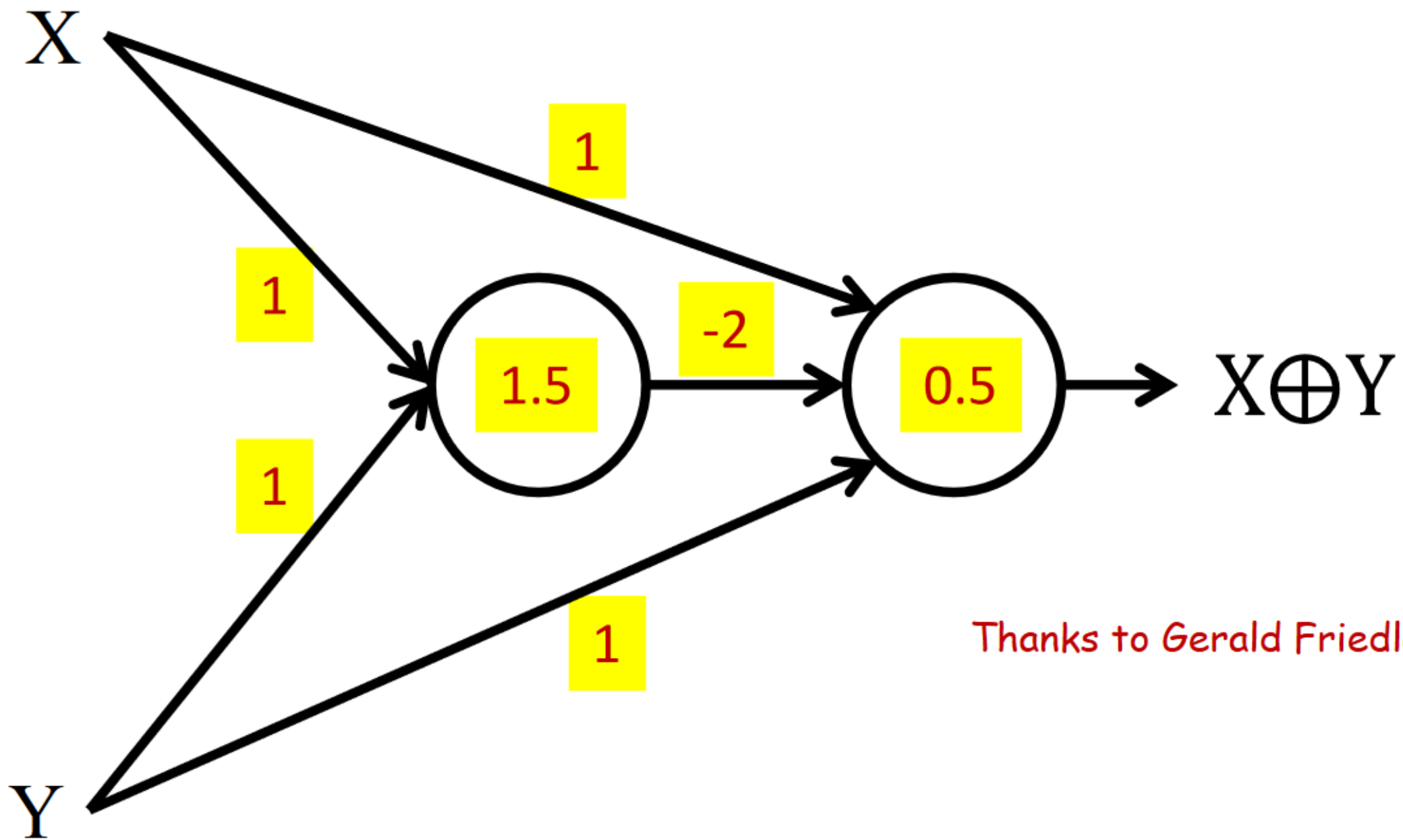
- A perceptron can model any simple binary Boolean gate

The perceptron is not enough



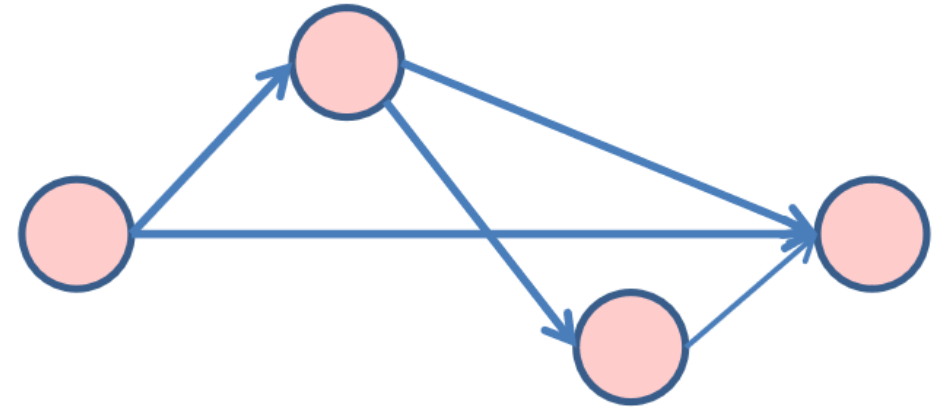
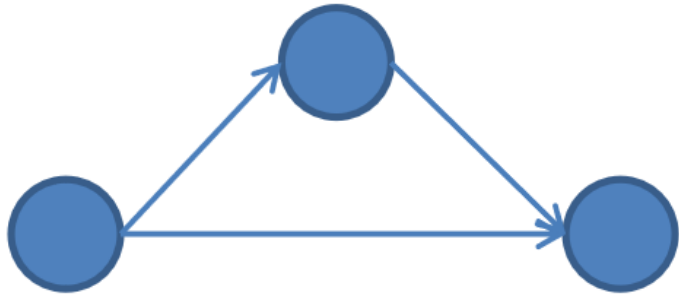
Cannot compute an XOR





Thanks to Gerald Friedland

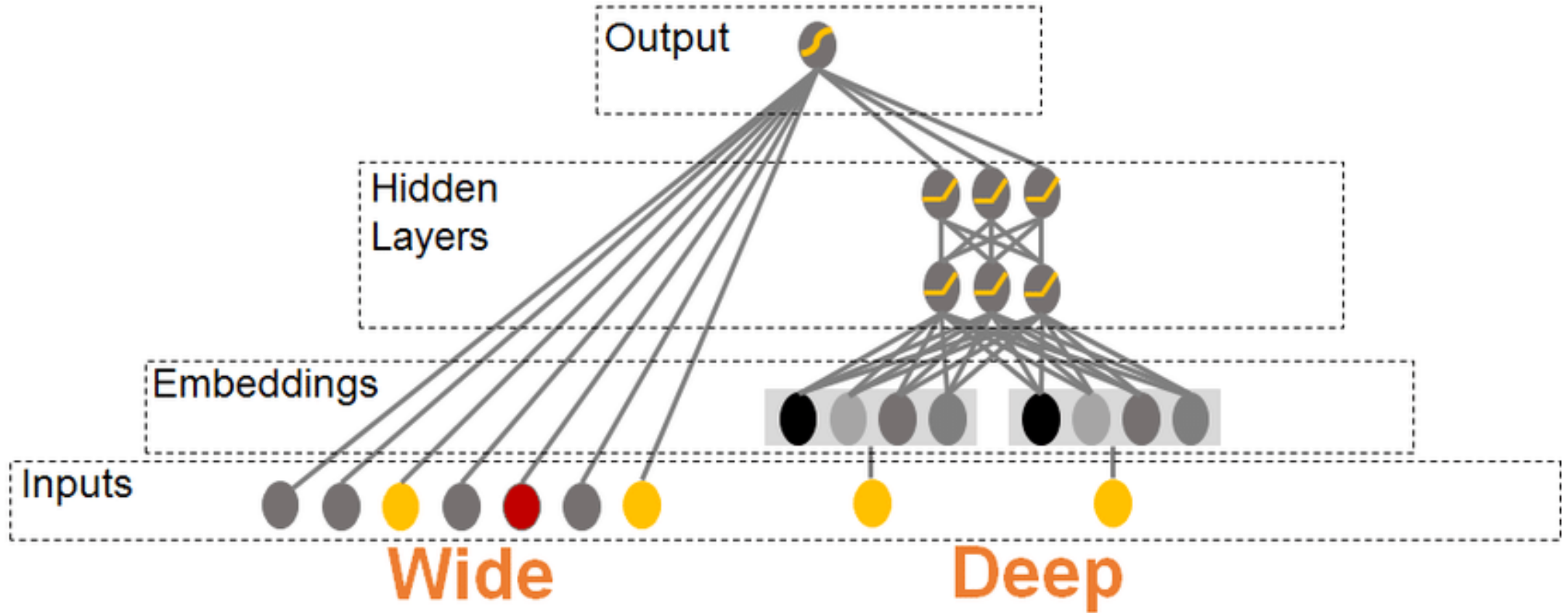
- In any directed network of computational elements with input source nodes and output sink nodes, “depth” is the length of the longest path from a source to a sink
 - A “source” node in a directed graph is a node that has only outgoing edges
 - A “sink” node is a node that has only incoming edges



• Left: Depth = 2.

Right: Depth = 3

Wide vs deep networks



Wide neural networks

- With enough width, the network can memorize training examples (especially in small or moderate datasets).
 - A sufficiently wide single-hidden-layer neural network with a nonlinear activation can approximate a large class of continuous functions.
 - Compared to very deep architectures, wide networks rely more on ample neurons in one or two layers rather than intricate multi-layer designs.
-
- ✓ A very wide network can memorize data easily, raising the likelihood of overfitting when the training set is not large or diverse.
 - ✓ Wide (shallow) networks, unlike deep architectures, do not learn hierarchically abstract features as effectively.

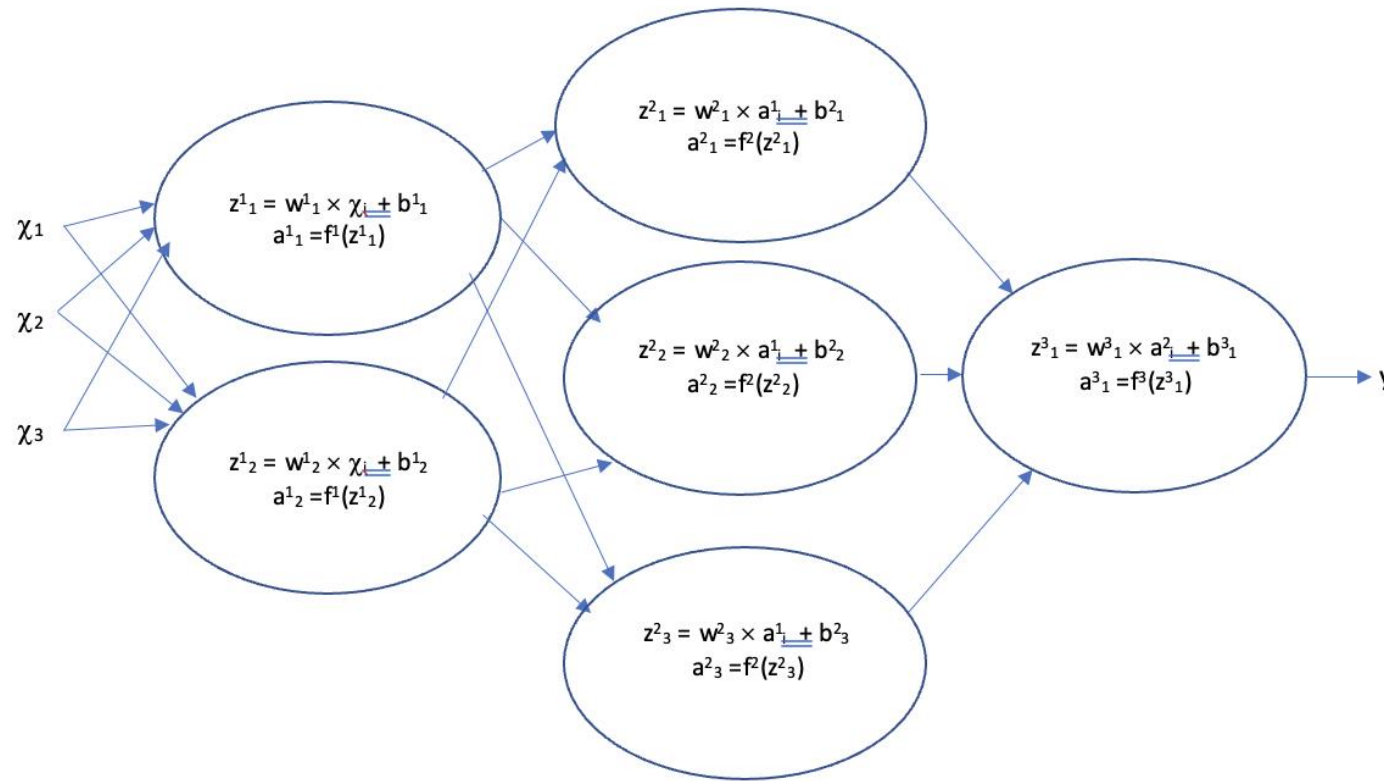
Memorization in wide networks

- Wide networks typically have a significantly larger number of trainable parameters. This high capacity allows them to learn detailed patterns—even idiosyncrasies of individual examples—rather than just capturing broader, generalizable trends.
- In high-dimensional parameter spaces, there are infinitely many solutions that can perfectly fit the training data. Wide networks can more readily stumble upon such solutions that memorize the training set rather than learning patterns that generalize well.

Deep neural networks

- In deep networks, lower layers tend to learn simple features (edges, corners, color blobs), while higher layers learn more complex patterns (object parts, semantic concepts). This hierarchical decomposition of features can lead to robust and generalizable representations.
- Deeper layers can gradually transform the input space into more compact, task-relevant feature spaces. This transformation helps remove irrelevant variations in the data (e.g., noise), focusing on features that matter for the target task.

Why is non-linearity important



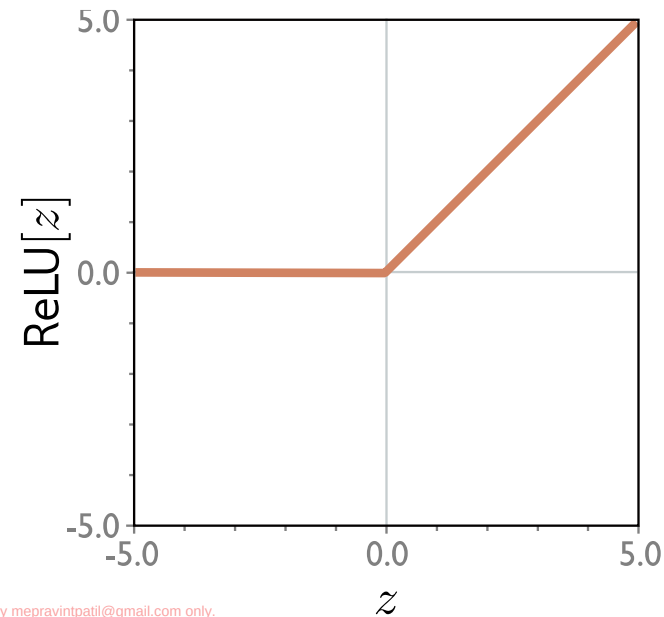
$$\begin{aligned}
 y &= f^3_1(z^3_1) = f^3_1(w^3_1 * a^2_1 + b^3_1) \\
 &= f^3_1(w^3_1 * f^2_1(z^2_1) + b^3_1) \\
 &= f^3_1(w^3_1 * f^2_1(w^2_1 * a^1_1 + b^2_1) + b^3_1) \\
 &= f^3_1(w^3_1 * f^2_1(w^2_1 * f^1_1(z^1_1) + b^2_1) + b^3_1) \\
 &= f^3_1(w^3_1 * f^2_1(w^2_1 * f^1_1(w^1_1 * x_1 + b^1_1)) + b^2_1) + b^3_1)
 \end{aligned}$$

Example shallow network

Activation function

$$\begin{aligned} y &= f[x, \phi] \\ &= \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x] \end{aligned}$$

$$a[z] = \text{ReLU}[z] = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}.$$



Example shallow network

$$\begin{aligned} y &= f[x, \phi] \\ &= \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x] \end{aligned}$$

$$\phi = \{\phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}\}$$

Hidden units

$$y = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x].$$

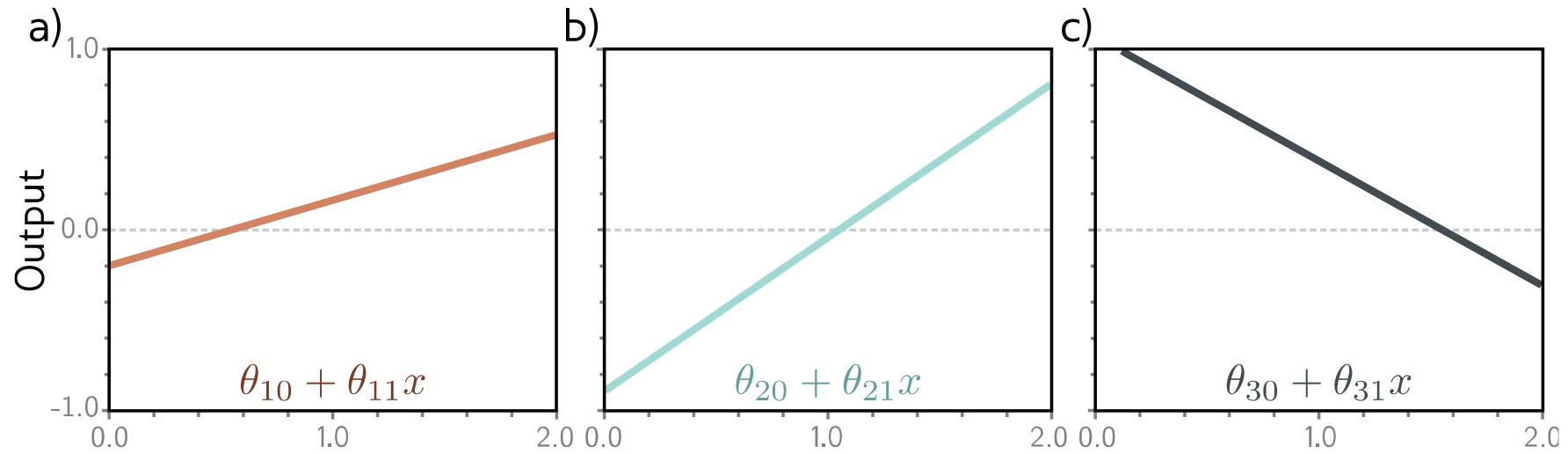
Break down into two parts:

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

where:

$$\text{Hidden units} \left\{ \begin{array}{l} h_1 = a[\theta_{10} + \theta_{11}x] \\ h_2 = a[\theta_{20} + \theta_{21}x] \\ h_3 = a[\theta_{30} + \theta_{31}x] \end{array} \right.$$

1. compute three linear functions

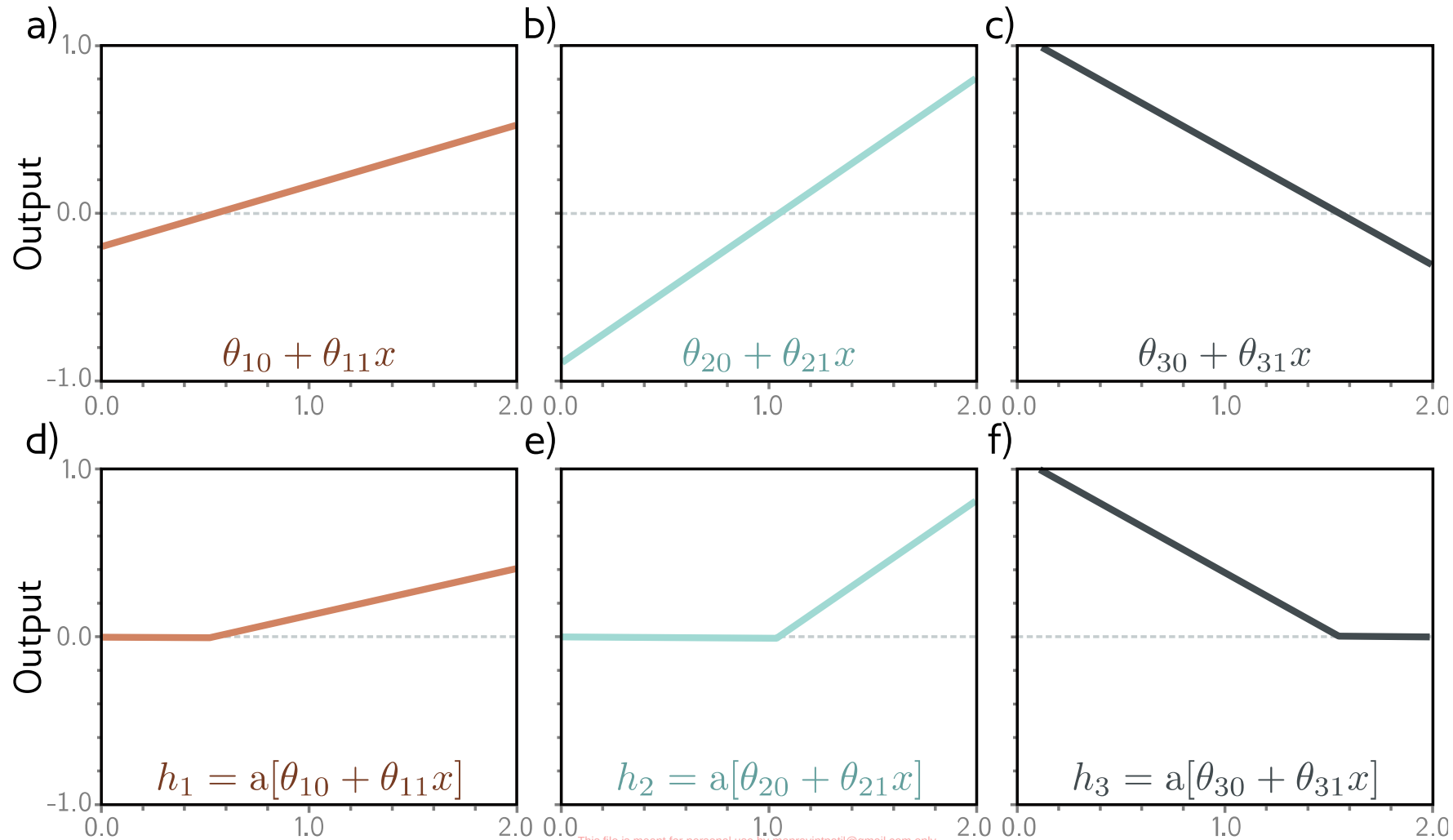


2. Pass through ReLU functions (creates hidden units)

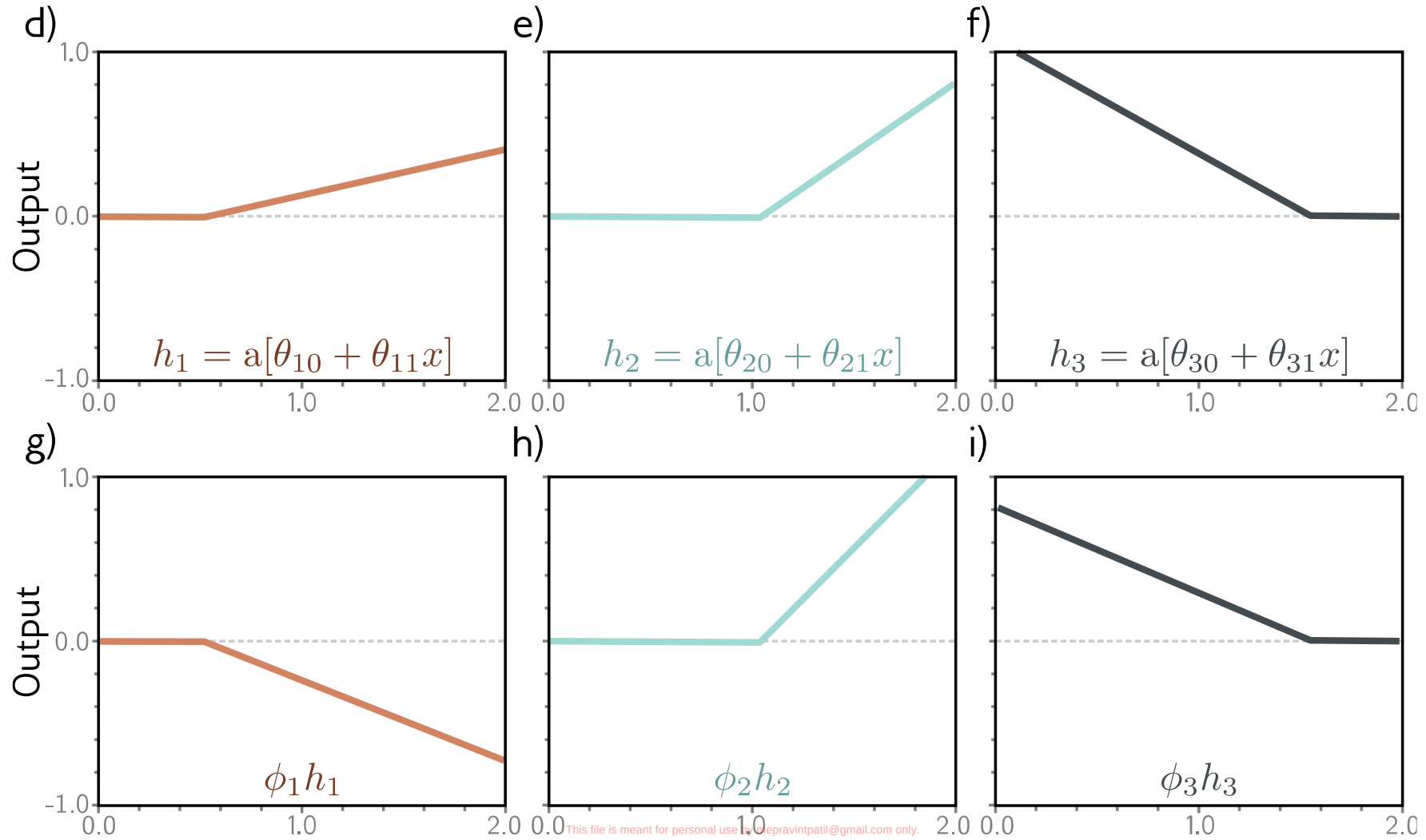
$$h_1 = a[\theta_{10} + \theta_{11}x]$$

$$h_2 = a[\theta_{20} + \theta_{21}x]$$

$$h_3 = a[\theta_{30} + \theta_{31}x],$$

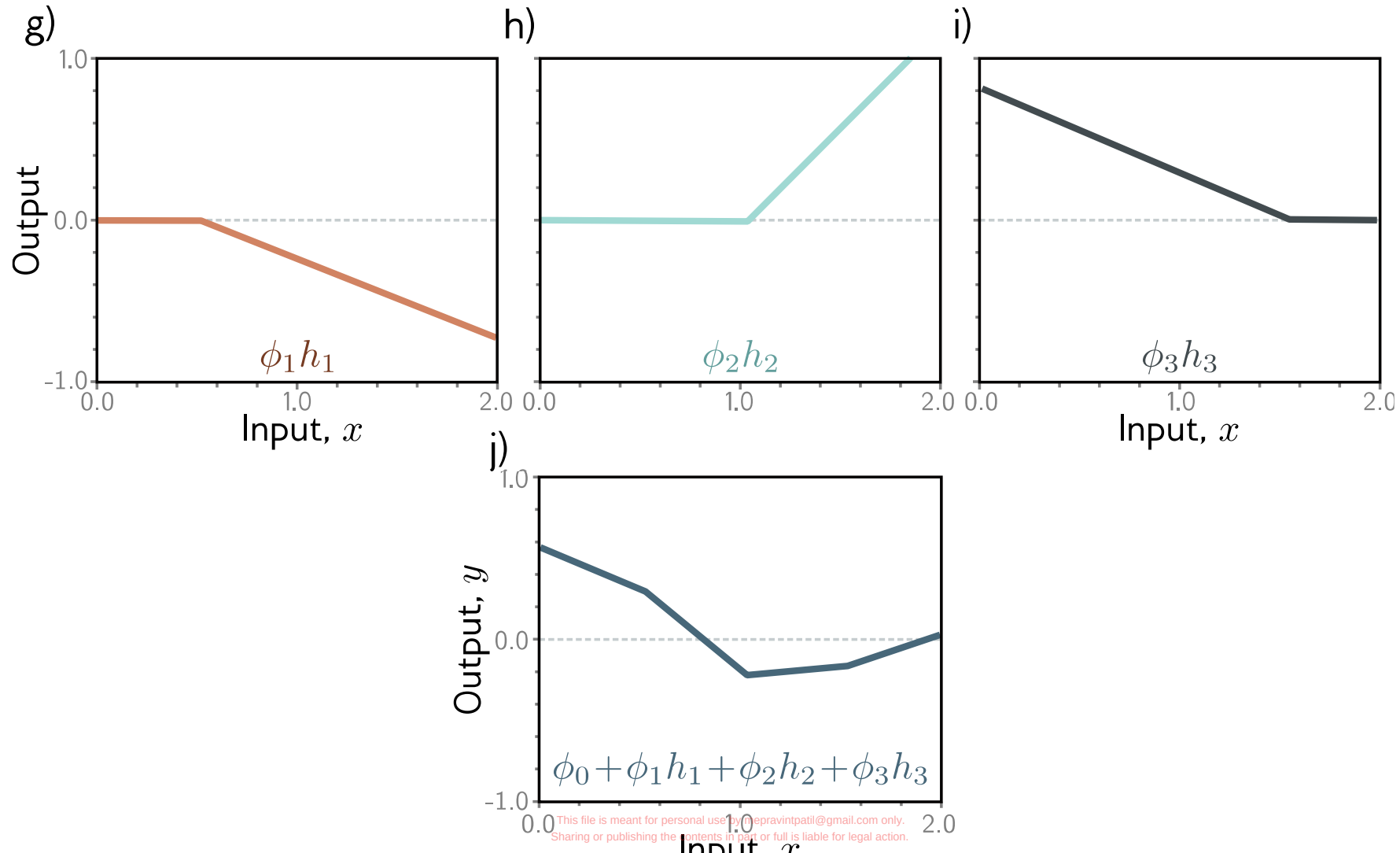


2. Weight the hidden units



4. Sum the weighted hidden units to create output

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$



With 3 hidden units:

$$h_1 = a[\theta_{10} + \theta_{11}x]$$

$$h_2 = a[\theta_{20} + \theta_{21}x]$$

$$h_3 = a[\theta_{30} + \theta_{31}x]$$

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

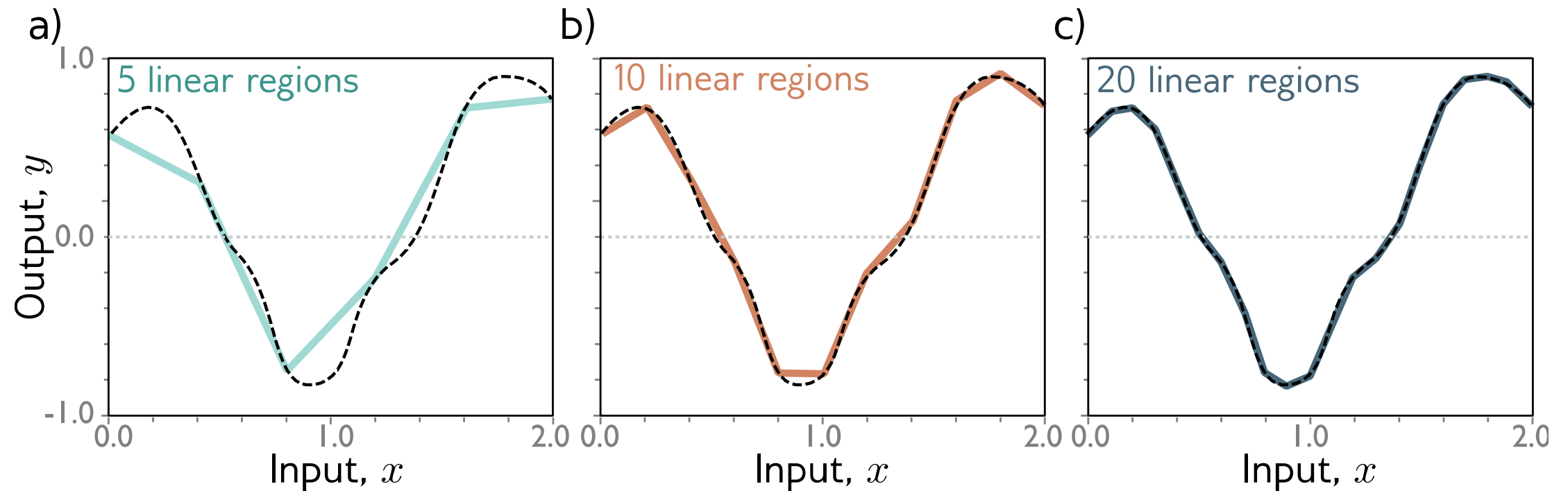
With D hidden units:

$$h_d = a[\theta_{d0} + \theta_{d1}x]$$

$$y = \phi_0 + \sum_{d=1}^D \phi_d h_d$$

With enough hidden units...

... we can describe any 1D function to arbitrary accuracy



Universal approximator theorem

- A feed-forward neural network with at least one hidden layer, a **sufficient number** of hidden units, and a **nonlinear** activation function can approximate **any** continuous function on a **compact** set to an arbitrary degree of accuracy.

Theorem's Emphasis: Approximation focuses on the ability to fit a function given perfect information (i.e., a known function or infinite data).

Real-World Problem: In practical machine learning, you have finite (often noisy) training data, so the model's ability to generalize beyond that data is **not** addressed by the universal approximation theorem.

A model could “approximate” a function well in theory but still *overfit* or fail to generalize.